A Dimensional Data Warehouse for Geospatial Monitoring of Municipal Public Works, with an Evolution Path Toward a Lakehouse Architecture

Uriel González Casiano[0009-0009-8172-9584], Marco Tulio Maldonado Mejía[0009-0005-4691-9607], and Gabriel Hurtado Avilés*[0009-0002-5686-1822]

Escuela Superior de Cómputo (ESCOM), Instituto Politécnico Nacional, Mexico City, Mexico
{ugonzalezc2400,mmaldonadom2200}@alumno.ipn.mx
ghurtadoa@ipn.mx
gonzalezcasianouriel@gmail.com, marcotuliomm.s45@gmail.com,
gabrielhuav@gmail.com

Abstract. The accelerated growth of public infrastructure data in municipal governments generates heterogeneous volumes that are difficult to manage with isolated spreadsheets or transactional systems. This article presents the design and prototype implementation of a dimensional Data Warehouse for the geospatial monitoring of municipal public works, complemented by object storage (Cloudflare R2) for unstructured evidence —photographic reports and PDF documents— linked bidirectionally to the warehouse. The warehouse adopts a star schema with ten dimensions (five Slowly Changing Dimension Type 2, two Type 1, three Type 0) and two fact tables, and exposes OLAP queries and five analytical views for delay detection, budget alerts, and citizen participation through a single REST API, released as reference code rather than as a hosted service. A published geospatial viewer offers graph-like exploration across works, communities, budgets and citizen proposals. Rather than claiming a full enterprise lakehouse, we deliberately scope the contribution as a warehouse-centric design and outline its evolution toward a lakehouse (an open table format —Iceberg/Parquet— queried directly over the object store) as future work. The prototype was exercised over a seeded synthetic dataset of 1,247 public works from 55 communities and evaluated through a documented, seeded protocol for the analytical detection module, which recovers 84.1% of the injected anomalies with 99.5% precision. The design enables transparent, auditable, and citizen-oriented oversight of public works without the infrastructure costs of enterprise platforms.

Keywords: Data Warehouse · Dimensional Modeling · Slowly Changing Dimensions · Geospatial Monitoring · Open Government Data · Public Works · Object Storage · Lakehouse Architecture

1 Introduction

Transparency in the management of municipal public works is one of the most persistent challenges of digital governance in Mexico. Public-works departments simultaneously manage contractors, budgets, field supervisors, progress photographs, monthly reports,

* Corresponding author: gabrielhuav@gmail.com

and hand-over processes; yet most municipalities operate with distributed file systems, spreadsheets, or, at best, isolated transactional systems that preclude historical analysis, early delay detection, and informed citizen participation.

The *lakehouse* paradigm [1] offers a reference response to this fragmentation: it unifies the flexibility of Data Lake object storage (unstructured data, direct ingestion, low cost) with the analytical capability of a Data Warehouse (star schema, slowly changing dimensions, OLAP views). This work does not implement a full enterprise lakehouse: given the data volume at municipal scale, we build a *dimensional Data Warehouse* as the analytical core and use object storage only as a repository of unstructured evidence linked to it. Full lakehouse convergence —an open table format queried directly over the lake— is proposed as an evolution path (Section 8).

This article presents a prototype for the geospatial monitoring of public works in Temascaltepec, State of Mexico (64,844 inhabitants across 55 communities).

Research Questions

The work is guided by three research questions, which structure the design and the evaluation:

RQ1. Can a dimensional Data Warehouse deployed on commodity managed services (PostgreSQL/Supabase plus S3-compatible object storage) provide auditable historical traceability of municipal public works at the scale of a rural Mexican municipality, without the enterprise platforms normally associated with lakehouse architectures?

RQ2. Which dimensional design decisions —fact grain, choice of Slowly Changing Dimension type per entity, and view materialisation strategy— are required so that delay detection, budget alerting and citizen participation can all be served from a single analytical schema?

RQ3. How effective is a threshold rule expressed over the warehouse's analytical views at recovering anomalies in public-works execution, compared with an unsupervised baseline, and which anomaly classes does it fail to recover?

RQ1 is addressed by the architecture and warehouse design (Sections 3 and 4) and by the cost and deployment discussion (Section 8); RQ2 by the grain, SCD and view-strategy analysis (Section 4); and RQ3 by the evaluation of the detection module (Section 7.4).

Contributions

The main contribution is threefold: (1) a reference architecture centered on a dimensional *Data Warehouse* with slowly changing dimensions (SCD 0/1/2), complemented with S3-compatible object storage for unstructured evidence and exposed through a public/private REST API (RQ1); (2) an interactive geospatial map that acts as a graph-like exploration interface, linking public works, communities, budgets, supervisors, construction companies, and citizen proposals (RQ2); and (3) a reproducible, non-circular evaluation of the analytical detection module against an independently injected ground truth, together with a failure analysis by anomaly class (RQ3).

Over a seeded test scenario, the prototype organizes 1,247 simulated public works with an accumulated budget of $127.4 million pesos (Section 7 details the full population). All figures reported in this article were obtained over synthetic data generated with a fixed seed; they characterise the behaviour of the system, not the municipality of Temascaltepec, and no claim about real public-works execution in that municipality is made or implied.

The remainder of the article is organized as follows: Section 2 reviews related work and contrasts the proposal with comparable systems; Section 3 describes the architecture; Section 4 details the warehouse; Section 5 describes the object storage layer; Section 6 presents the geospatial map; Section 7 reports results; and Section 8 concludes.

2 Related Work

2.1 Data Lakehouse and Architecture Convergence

Zaharia et al. [1] coined the term *lakehouse* to describe an architecture that combines the open, flexible storage of a Data Lake with the ACID transaction guarantees, schema control, and analytical performance of a Data Warehouse. The transactional layer that makes this convergence possible was described by Armbrust et al. [9], who show how an open table format over cloud object stores can provide ACID semantics and time travel without a dedicated database engine; this is precisely the mechanism our object layer does not yet implement. Databricks [3] extended this model to the geospatial domain, integrating vector and raster data on a unified platform.

In the present work we adopt the spirit of the lakehouse without requiring Apache Spark or Delta Lake: S3-compatible object storage (Cloudflare R2) acts as the evidence layer and PostgreSQL/Supabase as the warehouse layer, connected by a REST API and by database triggers rather than by an external ETL process. We stress that, in its current state, the object store is a binary-evidence repository and not a queryable analytical lake: no open table format (Iceberg/Delta/Hudi) and no engine querying over the lake are used. We therefore position the system as a dimensional Data Warehouse with object storage, and reserve full lakehouse convergence for future work.

2.2 Geospatial Knowledge Graphs

Knowledge graphs (KGs) have proven useful for integrating heterogeneous government data. Hogan et al. [4] provide a comprehensive survey of the state of the art in KGs, highlighting their application to georeferenced entities. Janowicz et al. [5] present KnowWhereGraph, one of the largest public geospatial graphs, which links human and environmental data under FAIR principles. In the open-government-data domain, Espinoza-Arias et al. [6] document the case of Zaragoza, where a municipal knowledge graph connects public services, infrastructure, and demographic data. Rajabi et al. [8] systematize the construction of knowledge graphs over open government data using Semantic Web technologies.

Our geospatial map adopts an analogous *relational* approach at a rural municipal scale: public works, communities, budgets, and citizen proposals constitute the entities, while reports and photographs represent linked evidence. The distinction with respect to the works above is deliberate and is stated explicitly in Section 6: the graph is implicit in the foreign keys of the dimensional model and is materialised only at the interface

level. Publishing the schema as an OWL vocabulary, which would place the system in the same family as [5,6,8], is the first item of our future work.

2.3 Open Government Data and Infrastructure Disclosure Standards

The literature on Open Government Data (OGD) [7] emphasizes that publishing public-infrastructure data in reusable formats increases accountability and facilitates the detection of irregularities. However, most OGD systems in Latin American municipalities are limited to static download portals, without analytical capability or integration with field data.

A directly relevant body of practice is the family of infrastructure disclosure standards. The Open Contracting for Infrastructure Data Standard (OC4IDS) [10], developed by the Open Contracting Partnership with CoST — the Infrastructure Transparency Initiative, defines a JSON schema for publishing project- and contract-level information across the infrastructure lifecycle. OC4IDS and our proposal are complementary: OC4IDS specifies *what* a government should disclose and in which interchange format, whereas we address *how* a small municipality can produce, historise and analyse that information internally. A natural convergence (Section 8) is to expose the warehouse as an OC4IDS-conformant endpoint, with `dim_work` as the project entity and `fact_audit_events` as the source of lifecycle transitions.

This same approach of consolidating official open data into a queryable analytical artifact has been applied before in other government domains in Mexico, such as seismic risk [11], a methodological antecedent transferred here to the monitoring of public works.

2.4 Positioning with Respect to Comparable Systems

Table 1 contrasts the proposal with representative systems along the dimensions that matter for municipal public-works oversight: the underlying data model, whether entity history is preserved, whether unstructured field evidence is a first-class citizen, whether a geospatial interface is provided, whether a formal semantic layer exists, and the deployment scale targeted.

Two gaps emerge. The semantic-web systems [5,6,8] provide a formal layer that we lack, but none historises entity state at audit granularity ("what did this record look like the day the budget was modified?"), which is what SCD 2 provides. And none links binary field evidence bidirectionally to the analytical store: in OGD portals and OC4IDS implementations, documents are referenced by URL without a managed lifecycle. This work fills the intersection of *temporal auditability* and *evidence linkage* at a scale where enterprise platforms are not economically viable.

3 System Architecture

Figure 1 illustrates the four layers of the proposed architecture, which follows a layered pattern with a clear separation of responsibilities. It is described as a reference architecture and prototype: the analytical core is the dimensional Data Warehouse, and the object store operates as an evidence layer. The subsections below correspond one-to-one to the four layers labelled in the figure, so that the terminology of figure and text coincides.

Table 1. Comparison with representative systems. SCD 2 = Slowly Changing Dimension Type 2; "Ev." = managed bidirectional link between binary field evidence and the analytical store; "Sem." = semantic layer.

System	Data model	History	Ev.	Geo UI	Sem.
KnowWhere Graph [5]	RDF knowledge graph	Snapshot	No	Yes	OWL/ RDF
Zaragoza KG [6]	RDF knowledge graph	Partial	No	Partial	Ontology
Nova Scotia OGD KG [8]	RDF from OGD sets	No	No	No	Ontology
OC4IDS [10]	JSON disclosure schema	Lifecycle stages	By URL	Portal-dep.	JSON Schema
Geospatial lakehouse [3] vector	Delta/Parquet tables Via BI	Table versions None	Raster/		
Typical LatAm OGD portal	CSV/PDF downloads	No	No	Rare	None
This work	Dimensional star schema	**SCD 2 (5 dims)**	**Yes**	**Yes**	None (future work)

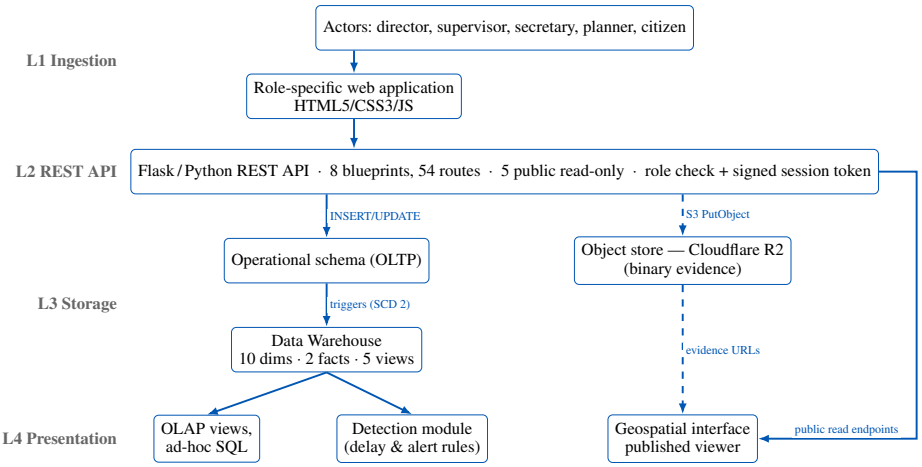

Fig. 1. System architecture (warehouse-centric), redrawn in English. The four layers L1–L4 correspond to Sections 3.1–3.4. Solid arrows: structured-data flow; dashed arrows: unstructured (binary) evidence flow. Released role-specific pages read operational endpoints, while the warehouse serves detection and OLAP; the geospatial prototype is not a released artefact.

3.1 Layer 1: Ingestion

The actors (directors, supervisors, secretaries, planners and registered citizens) interact with a web application built in HTML5/CSS3/JavaScript. Forms send structured data to the API (text, coordinates, amounts) and binary files (JPG/PNG, PDF) to the same endpoint, which routes them to their destination through an atomic transaction.

3.2 Layer 2: REST API

A Flask/Python REST API organised in eight blueprints exposes 54 routes, of which five are public and read-only (`/api/public/obras`, `/api/public/regiones`,

`/api/public/resumen` and two evidence-listing routes). Access control for municipal staff is performed by a `@require_auth` decorator that validates a role identifier transmitted in request headers; the participatory-budget module for citizens uses HMAC-signed session tokens issued with `itsdangerous` over the application secret. Passwords are stored as PBKDF2-HMAC-SHA256 hashes with a 16-byte random salt. We describe the scheme precisely because it is deliberately lightweight and is *not* equivalent to a full token-based authentication service: hardening it (token expiry, rotation, and moving the staff role check from a header to a signed token) is listed among the limitations in Section 8.2.

3.3 Layer 3: Storage

Object store (Cloudflare R2). An S3-compatible object store that receives unstructured evidence through direct ingestion. Paths follow a *hive-partitioned* convention, shown in Equation (1):

```
works/{work_id}/reports/{year}-{month}/{ts}_{slug}.{ext}    (1)
```

This structure enables direct exploration by work and period; each object's metadata (public URL, original name, MIME type, size in bytes, upload date) is also persisted in a warehouse-side table, creating a bidirectional link between the two layers.

Data Warehouse (PostgreSQL/Supabase). An analytical layer with a star schema, two fact tables, and ten dimensions (Section 4). The warehouse is populated from the operational schema by database triggers rather than by an external ETL job: eight triggers synchronise the dimensions (applying SCD 2 versioning where required) and four triggers emit rows into the audit fact table when a progress report, a cost record, an image upload or a citizen vote is registered. The warehouse uses range partitioning by year on the audit fact table to optimize historical queries.

3.4 Layer 4: Presentation

The released presentation layer consists of role-specific HTML/CSS/JavaScript pages that consume the REST API. Analytical consumption —the five views of Section 4.3, ad-hoc OLAP queries and the detection module of Section 3.5— is served directly from the warehouse. The geospatial viewer of Figure 3 is published alongside the rest of the artefact and reads a frozen projection of the same synthetic dataset, so map and workspaces never disagree.

3.5 Analytical Detection Module

The component evaluated in Section 7.4 is a stateless component that reads the warehouse and emits two artefacts: a *delay set*, from `v_delayed_works`, containing every work whose latest monthly fact row carries the delay flag together with its slippage; and an *alert stream*, from `v_audit_alerts`, which classifies audit events into five categories through a `CASE` expression over the fact table joined with the time, event-type and work dimensions.

The rule applied to a work–period record flags it as anomalous if *any* of three operational criteria holds:

Table 2. Warehouse dimensions and the type of SCD implemented.

Dimension	Modeled entity	SCD
`dim_time`	Calendar period	0
`dim_work`	Public work	2
`dim_region`	Community/neighborhood	2
`dim_company`	Construction company	2
`dim_staff`	Public-works department staff	2
`dim_source`	Funding source	0
`dim_budget`	Budget line	2
`dim_citizen`	Registered citizen	1
`dim_proposal`	Citizen proposal	1
`dim_event_type`	Catalog of 17 event types	0

C1. *Cost outlier*: the executed budget deviates from the mean by more than three standard deviations, $|b_i - \mu_b|/\sigma_b > 3$.
C2. *Extreme delay*: the recorded slippage exceeds 120 days.
C3. *Progress inconsistency*: physical progress below 30% together with financial progress above 80%.

The corresponding anomaly score is $s_i = \max\left(\frac{|b_i-\mu_b|}{\sigma_b},\ \frac{d_i}{120}\mathbb{1}[d_i > 120],\ 3.5 \cdot \mathbb{1}[\text{C3}]\right)$, where d_i is the slippage in days; the module flags $s_i > 3$. The score is used for the ranking-based metrics reported in Section 7.4. The alert stream is designed for presentation through the API.

3.6 A Note on Identifiers

This article uses English identifiers for tables, columns and views; the published implementation uses Spanish ones, as it was built with and for a Spanish-speaking municipal office. The correspondence is mechanical —`fact_audit_events` is `fact_eventos_auditoria`, `v_delayed_works` is `v_obras_retraso`, `dim_work` is `dim_obra`, and so on— and the repository README tabulates every pair, so that a reader inspecting the code can locate each object referenced here. This mapping is a prerequisite for the reproducibility claims of Section 7.

4 Data Warehouse Design

The warehouse adopts a dimensional star model [2] with two fact tables and ten dimensions (Table 2), shown schematically in Figure 2.

4.1 Fact Tables and Grain

We state the grain of each fact table explicitly, since it determines every subsequent aggregation and is a prerequisite for reproducing the design.

`fact_audit_events` has a grain of *one row per audit event*: a single, atomic, immutable action performed on a work by an identified actor at an identified instant. It

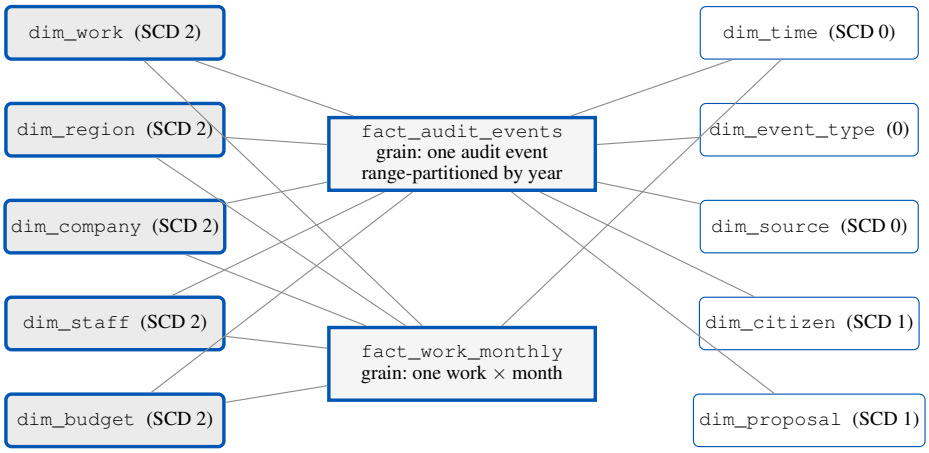

Fig. 2. Dimensional star schema, redrawn for legibility. Two fact tables (`fact_audit_events`, range-partitioned by year, and `fact_work_monthly`) are connected to the ten dimensions through surrogate keys. The five Slowly Changing Dimensions of Type 2 (shaded, left) are those that carry the historical traceability of the works. The monthly fact table is conformed only with the five SCD 2 dimensions and the time dimension.

records events classified into 17 types (work creation, work modification, state change, progress report, cost record, budget record, budget modification, image upload, permit issue, hand-over record, contractor selection, funding assignment, supervisor assignment, role change, proposal registration, vote cast and session start) and is partitioned by year. It contains 8,934 events (test data) with foreign keys to the ten dimensions, financial metrics, progress metrics, and audit flags.

`fact_work_monthly` has a grain of *one row per work per calendar month*. It is a periodic snapshot table: for each work and month it stores the total budget, the accumulated cost, the remaining balance, the percentage executed, the physical progress, the number of reports and images registered, and a boolean delay flag with the associated slippage in days. Because the grain is a snapshot rather than a transaction, aggregating cost across months requires taking the last snapshot of the period rather than a sum; the analytical views encapsulate this rule so that consumers cannot get it wrong.

4.2 Slowly Changing Dimensions

The SCD 2 dimensions [2] (five in total: works, regions, companies, staff, and budget) maintain history through validity columns (`effective_date`, `expiration_date`, `is_current`), which makes it possible to reconstruct the exact state of any work at an arbitrary point in time.

Three representative use cases motivate this choice, none answerable with a Type 1 dimension. In a *budget-modification audit*, `dim_budget` closes the current version and opens a new one, so an auditor can ask which amount was in force on the date a payment was authorised, not merely the amount in force today. Under *contractor reassignment*, `dim_company` preserves both versions, so progress recorded before the change stays attributed to the original contractor —a precondition for any performance analysis by contractor. Under *territorial reorganisation*, where communities are merged

or renamed, dim_region versioning prevents a historical investment series from being silently rewritten.

The remaining dimensions need no versioning: dim_time, dim_source and dim_event_type are fixed catalogues (Type 0), while dim_citizen and dim_proposal are Type 1 because only their current state is analytically meaningful and retaining superseded personal records would be undesirable.

4.3 Analytical Views and Refresh Strategy

Five analytical views derived from the fact tables support analytical consumption: (1) **v_delayed_works** (works with a schedule slip and days of delay); (2) **v_audit_alerts** (alerts classified as HIGH_AMOUNT, CANCELLED_WORK, BUDGET_CHANGE, LATE_EVENT and REVIEW); (3) **v_work_traceability** (full SCD 2 version history of a work); (4) **v_citizen_participation** (votes and proposals by region); and (5) **v_budget_executed** (accumulated budget execution by funding source).

These are standard SQL views rather than materialized ones. The choice is deliberate and determines the refresh strategy. At the volumes involved —of the order of 10^4 fact rows for 64,844 inhabitants— the views execute in tens of milliseconds over the partitioned fact table, so materialisation would buy little while introducing a staleness window; and since alerts must be actionable the moment a supervisor uploads a report, a live view is semantically preferable, with no refresh job to schedule or to fail. Should the deployment scale to several municipalities, the migration path is to materialise the aggregation-heavy v_budget_executed and v_citizen_participation with REFRESH MATERIALIZED VIEW CONCURRENTLY after each nightly load, keeping the two alert-bearing views live.

5 Object Storage Layer: Evidence Storage

The object layer uses Cloudflare R2, a service compatible with the Amazon S3 API, selected for its zero egress cost and reduced global latency. In its current state it functions as an object repository for binary evidence (images and PDFs) linked to the warehouse, and not as a queryable analytical lake: analytical queries are resolved in the Data Warehouse, not over R2. Adopting an open table format [9] that enables direct queries over the lake is proposed as future work (Section 8). The path design follows the *hive-partitioned* convention [3] of Equation (1).

Ingestion is direct from the web form: the supervisor attaches images or PDFs when creating a progress report; the API computes the object key, uploads it through the S3 SDK, records the metadata in the warehouse, and returns the public URL, all in a single atomic transaction. If the PostgreSQL insert fails, the upload to R2 is rolled back.

6 Geospatial Interface Prototype

The geospatial display is an *implicit* graph at the interface level —the relationships are derived from the foreign keys of the dimensional model— and not a formal knowledge graph with an ontology or an RDF triple store; publishing the schema as a linked vocabulary is addressed in future work.

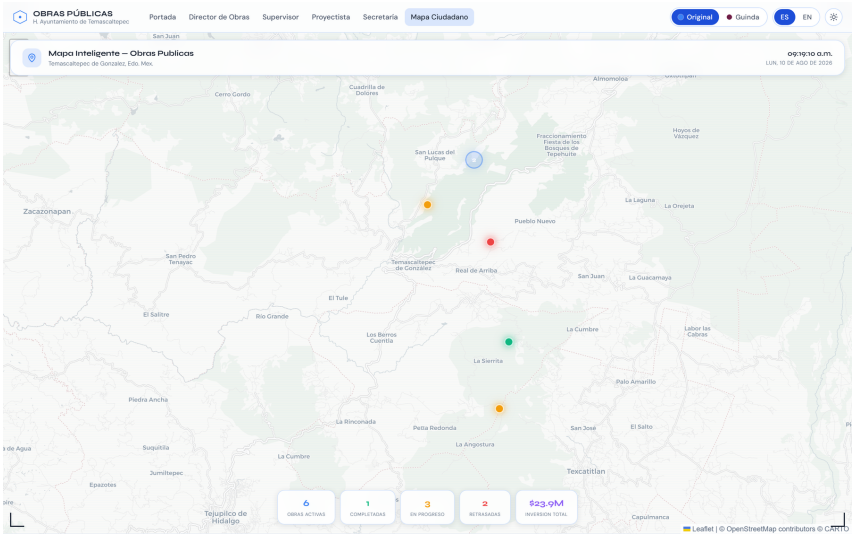

Fig. 3. The published geospatial viewer, in light mode. Markers are color-coded by state —green completed, amber in progress, red delayed— over the six works of the demonstration dataset, with the indicator panel below. Live at https://gabrielhuav.github.io/PublicMunicipalWorks_DWH/mapa/?modo=claro, which reproduces this exact view.

Figure 3 shows the main interface, with markers color-coded by the state of the work and the technical-card popup that displays physical and financial progress indicators when a node is selected. It is part of the released artefact, reachable from the navigation bar of every page. The warehouse holds no surveyed coordinates, so each marker sits at a deterministic position hashed from its community and work identifier inside the municipal bounding box: stable across reloads, but not the real location of anything.

6.1 Management and Visualization Components

- **Marker layer:** each active work is represented by a marker color-coded by state —green (completed), amber (in progress), red (delayed)—derived from the work's latest progress report.
- **Technical card (popup):** selecting a work displays its file number, name, community, total budget, assigned company, date range, a physical-progress bar, and an expandable section for images from the object layer.
- **Analysis layer:** highlights delayed works and indicates the days of slippage (criterion C2 of Section 3.5); the dual physical/financial percentage detects works with high budget execution and low physical progress (criterion C3), a potential indicator of irregularities.

7 Results and Evaluation

7.1 Scope of the Evaluation

Table 3. Lighthouse audit of the published artefact (median of three runs for the landing page). Reproducible over the public URL.

	Core Web Vitals					Scores (0–100)			
Page	FCP	LCP	SI	TBT	CLS	Perf.	A11y	B.P.	SEO
Landing	3.2 s	3.2 s	4.9 s	0 ms	0.082	83	90	96	90
Map	2.6 s	5.4 s	3.0 s	80 ms	0.006	76	96	93	100

Measured results, design targets and projections are kept strictly apart in what follows. We label every figure in this section as **measured** (obtained by executing published code or auditing the published artefact, and reproducible by a third party: Sections 7.3 and 7.4), **target** (derived from architectural sizing and spot measurements of individual components, never from an end-to-end benchmark of a deployed system: the latency figures of Section 7.2), or **projected** (an expected operational outcome not yet observed in any deployment: Section 7.5). No number in this article is a measurement taken on real municipal data.

7.2 Performance Targets (Design, not Measured)

The design targets for a reference configuration (50 concurrent users, 95th percentile) are: 187 ms response time on `/api/public/obras`; 1.42 s initial map load; 543 ms for a four-image gallery; 127 req/s sustained; 89 ms to execute `v_delayed_works`; 34 ms for an insert firing the SCD 2 trigger; and 112 ms read latency from the object layer. They derive from architectural sizing and spot measurements on individual components, are targets in the sense of Section 7.1, and must not be read as a benchmark: none is a measurement of a deployed system, and the Flask+Supabase+R2 backend is distributed as a reference implementation whose permanent deployment is not guaranteed. The test population reproduces the intended operating structure: 1,247 works across 55 communities, 8,934 audit events, 3,421 photographic pieces of evidence, $127.4 million pesos in budget, 2,156 citizen proposals, and 8,723 votes cast.

7.3 Evaluation of the Published Frontend (Measured)

The static artefact, unlike the backend, is permanently online, so its delivery can be audited by anyone against a public URL. Table 3 reports Google Lighthouse over the two entry points: the landing page and the vendored geospatial viewer. These are measurements in the sense of Section 7.1, not targets.

The two pages fail in opposite ways. The landing page is delayed by render-blocking web fonts and a CDN animation library, yet leaves the main thread idle (0 ms blocking); the map barely delays first paint but pays for a 533 kB bundle with a 5.4 s largest element. Neither is a property of the warehouse: both are delivery traits of an unoptimised front end, remediable without touching the architecture. Lighthouse audits *page delivery*, not query latency; the latter is in Section 7.2 and needs the deployed backend.

Conditions: Lighthouse 12.8.2, headless Chrome 151, emulated mobile device with simulated throttling, 10 August 2026. The artefact is static and versioned, so re-running the audit yields the same page that produced these figures.

Table 4. Detection performance against an independently injected ground truth (synthetic dataset, 1,421 records, seed 42). Reproducible with the published script.

Method	Precision	Recall	F1	ROC-AUC	PR-AUC
Threshold rule (this work)	0.9945	0.8411	0.9114	0.9356	0.8702
Isolation Forest (baseline)	0.8545	0.8505	0.8525	0.9855	0.9289

7.4 Evaluation of the Delay/Alert Detection Module (Measured)

Ground truth. Deriving the ground truth from the same predicate used by the detector would make precision equal to 1 by construction and the comparison uninformative. We therefore take the ground truth from an independent source: the data generator injects anomalies, with probability 0.15 and in four classes —*cost overrun* (the executed budget is multiplied by a factor drawn from $[1.5, 3.0]$), *delay* (slippage drawn from $[120, 365]$ days), *inconsistency* (physical progress forced to $[5, 25]\%$) and *ghost work* (physical progress forced to 0)— and records the class in a label field that the detector never observes. That label is our ground truth. Over the seeded dataset, 214 of 1,421 work–period records (15.06%) are anomalous.

Setting. The dataset comprises 1,421 work–period records across the 55 communities of Temascaltepec, generated with seed 42. It is a distinct artefact from the 1,247-work warehouse dataset used elsewhere in this section, and the two must not be conflated. The detector is the rule of Section 3.5. The baseline is an Isolation Forest (100 estimators, contamination 0.15, seed 42) over five normalised features: executed budget, physical progress, financial progress, slippage and the community development index.

Explanation of the results. Table 4 shows a clear division of labour between the two methods, and the interesting content of the experiment is in that division rather than in the headline F1.

The threshold rule is almost perfectly precise (0.9945: a single false positive in 1,421 records) but recovers only 84.1% of the anomalies. Its 34 misses are not distributed uniformly across classes. Decomposing recall by injected class, the rule recovers 65/65 ghost works, 57/57 inconsistencies and 43/43 delays —all at recall 1.0— but only 15/49 cost overruns (0.3061). The reason is structural: criterion C1 flags a budget only beyond three standard deviations, whereas the injected overrun multiplies the budget by a factor of at most 3 over a distribution that is already right-skewed across heterogeneous work types. A moderate overrun on an already-expensive work therefore remains inside the 3σ envelope. This is a genuine and actionable design finding: a global σ threshold is the wrong instrument for cost anomalies, and the criterion should be computed *per work type* rather than over the whole population.

The Isolation Forest inverts the trade-off. Its recall is marginally higher (0.8505) but it produces 31 false positives against the rule's single one, and its precision drops to 0.8545. Its ranking quality is nevertheless clearly superior (ROC-AUC 0.9855 versus 0.9356; PR-AUC 0.9289 versus 0.8702), because it exploits the joint distribution of the five features rather than three independent axis-aligned cuts. For an oversight application the operating point matters more than the ranking: an alert panel that a municipal officer reviews manually is dominated by precision, since false alarms erode trust in the tool. The rule is therefore the better default, and the baseline indicates how much headroom a learned detector would offer if recall on cost overruns were the priority.

7.5 Illustrative Analytical Capability (Projected)

Methodological note. The results in this subsection were obtained over a synthetic dataset generated with Python's Faker library (seed 42), which replicates the dimensional structure and the volume the system would process in a real deployment in Temascaltepec. They demonstrate that the analytical pipeline runs end to end and produces the intended artefacts; they are not evidence about the municipality, and the patterns described below are properties of the generator, not findings.

What the architecture can compute. A controlled correlation was injected between the simulated socio-economic development index and the days of delay, and the pipeline recovers it ($\rho = -0.67$, $p < 0.01$, $n = 55$ communities). What this demonstrates is not the coefficient, which was placed there by construction, but that the system can compute Spearman correlations over dimensional data, join them to the region dimension, raise alerts and render them geospatially. A Pareto-style budget concentration analysis and a comparison of participation rates by connectivity were exercised in the same way. Whether such patterns exist in Temascaltepec is an empirical question this article does not answer.

Projected operational benefits. We state these qualitatively rather than numerically, since any point estimate would require a derivation that the present evidence does not support. The system is designed to reduce the time needed to assemble a monthly progress report, to lower the volume of formal information requests by making the same data publicly queryable, and to surface budget anomalies early enough to act on them. Quantifying any of these requires a pre/post measurement in the municipal office, the first item of the validation plan below.

8 Conclusions and Future Work

8.1 Conclusions

This work presented a pragmatic architecture centered on a dimensional Data Warehouse for the geospatial monitoring of municipal public works, complementing the analytical store with S3-compatible object storage for unstructured evidence under a single REST API. Returning to the research questions: RQ1 is answered affirmatively at the design level —ten dimensions, five of them SCD 2, two fact tables and five views on managed services at an estimated $47 USD/month for 64,844 inhabitants— with the caveat that no measurement on a stable production deployment exists yet. RQ2 is answered in Section 4: an event-grain transaction fact table plus a work-month snapshot table, with SCD 2 restricted to the five audit-relevant entities, suffices to serve delay detection, budget alerting and participation analytics from one schema. RQ3 is answered in Section 7.4: the rule attains 0.9945 precision at 0.8411 recall against an independent ground truth, with failures concentrated in a single anomaly class whose criterion is mis-specified.

The proposal shows that it is possible to build a transparency system with analytical capability (OLAP views, SCD 2 history), georeferenced photographic evidence, and citizen participation, without relying on high-cost enterprise platforms.

8.2 Limitations

(1) The dynamic deployment uses a free hosting tier whose database has limited validity and whose web service is suspended on inactivity; permanence is delegated to

the static version, with a planned migration to a provider that does not suspend the service. (2) The participation module uses a simple voting scheme that may not adequately represent minorities. (3) The quality of the analysis depends on the quality, coverage, and freshness of the data entered by field supervisors. (4) All analytical results come from synthetic data (Faker, seed 42), due to privacy constraints on the open publication of municipal data; no result in this article has been validated against real municipal records. (5) The architecture is, in its current state, *data-warehouse-centric*: the object store holds binary evidence but is not queried analytically. (6) The public map endpoints read the operational schema rather than the warehouse views, so the two read paths can diverge; unifying them is pending. (7) Access control is lightweight —the staff role travels in a request header and the citizen session token, though HMAC-signed, has no expiry— which is adequate for a prototype whose analytical surface is public by design, but must be hardened before handling non-public data.

8.3 Future Work

(0) Evolve the object layer toward a full lakehouse by adopting an open table format [9] (Apache Iceberg, or Parquet with transactional metadata) queryable directly over R2 through a lightweight engine such as DuckDB or Trino, without introducing Apache Spark; (1) a computer-vision inference pipeline to automatically classify work progress from the stored photographs; (2) publishing the schema as an OWL vocabulary to interoperate with Linked Data, and exposing an OC4IDS-conformant [10] projection of the warehouse so that the municipality's data becomes comparable with other CoST-aligned jurisdictions; (3) quadratic voting to improve the representativeness of minority proposals; (4) time-series models to predict completion dates; (5) recomputing the cost-anomaly criterion per work type, following the failure analysis of Section 7.4, and re-evaluating against the same ground truth; and (6) validating the architecture in 3–5 additional municipalities of the State of Mexico, with a pre/post measurement of report-preparation time and information-request volume.

8.4 Availability

The warehouse DDL, SCD 2 triggers, seeded generators, load-testing scripts and the evaluation script are archived at https://github.com/gabrielhuav/PublicMunicipalWorks_DWH. A permanently deployable static demonstration is published through GitHub Pages at https://gabrielhuav.github.io/PublicMunicipalWorks_DWH/. It preserves the interface of the original public-works prototype—its role-access, budgeting, supervision, records, citizen-participation and map views—replacing remote authentication and writes with synthetic in-browser behaviour, and adds Spanish/English switching and two colour themes in light and dark mode. It therefore makes no request to Flask, Python, Render, Supabase or a live API. Access is open by construction: the credentials are printed on the landing page and the form accepts any value, even none. Every screen is served from a synthetic dataset held in the visitor's own browser tab, so no input can affect another visitor. The Flask implementation in `backend/` remains the operational reference, where the same `DEMO-` accounts are constrained to read-only access. Table 4 is reproduced by running `generar_dataset_obras.py` and `eval_deteccion_v2.py` from `scripts/evaluacion/`, both with seed 42. The population figures of Section 7 are constants in `scripts/generate_`

`synthetic_data.py`, met by construction; its `--verificar` flag checks them without a database.

Declaration on the Use of Generative AI

The authors used a generative AI assistant, under their direction, for language editing, LaTeX formatting, drafting text subsequently revised and approved by the authors, and writing the auxiliary scripts distributed with the artefact. The research questions, the dimensional design, the ETL, the data collection, the experiments and the interpretation of the results are the authors' own work.

Disclosure of Interests

The authors declare that they have no conflicts of interest relevant to the content of this article.

References

1. Zaharia, M., et al.: Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. In: Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR 2021). Online (2021)
2. Kimball, R., Ross, M.: The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd edn. Wiley, Indianapolis (2013)
3. Palkar, S., Thusoo, A., Boncz, P.A.: Building a geospatial lakehouse, part 1. Databricks Engineering Blog (2021). https://www.databricks.com/blog/2021/12/17/building-a-geospatial-lakehouse-part-1.html. Last accessed 15 Jan 2026
4. Hogan, A., et al.: Knowledge graphs. ACM Comput. Surv. **54**(4), Article 71 (2021). https://doi.org/10.1145/3447772
5. Janowicz, K., et al.: Know, know where, KnowWhereGraph: a densely connected, cross-domain knowledge graph and geo-enrichment service stack for applications in environmental intelligence. AI Mag. **43**(1), 30–39 (2022). https://doi.org/10.1002/aaai.12043
6. Espinoza-Arias, P., Fernández-Ruiz, M.J., Morlán-Plo, V., Notivol-Bezares, R., Corcho, O.: The Zaragoza's knowledge graph: open data to harness the city knowledge. Information **11**(3), 129 (2020). https://doi.org/10.3390/info11030129
7. Lourenço, R.P.: An analysis of open government portals: a perspective of transparency for accountability. Gov. Inf. Q. **32**(3), 323–332 (2015). https://doi.org/10.1016/j.giq.2015.05.006
8. Rajabi, E., Midha, R., de Souza, J.F.: Constructing a knowledge graph for open government data: the case of Nova Scotia disease datasets. J. Biomed. Semant. **14**, 4 (2023). https://doi.org/10.1186/s13326-023-00284-w
9. Armbrust, M., et al.: Delta Lake: high-performance ACID table storage over cloud object stores. Proc. VLDB Endow. **13**(12), 3411–3424 (2020). https://doi.org/10.14778/3415478.3415560
10. Open Contracting Partnership, CoST — the Infrastructure Transparency Initiative: Open Contracting for Infrastructure Data Standard (OC4IDS). https://standard.open-contracting.org/infrastructure/latest/en/. Last accessed 1 Aug 2026
11. Villa Vargas, J.M., Hurtado Avilés, G., Climent Hernández, J.A.: Cuando México tiembla: la historia contada por los datos. AZCATL Rev. Divulg. Cienc. Ing. Innov. **4**(6), 28–33 (2026). https://doi.org/10.24275/AZC2026E1004